

VisionInspect AI

Frontend Development – Milestone 2 Report
Himabindhu Ravuri

Infosys Springboard Internship

1. Project Description

VisionInspect AI is a system that checks product images for defects using computer vision, and tells us whether a product passes or fails quality check. It's being built by our 5-member team as part of the Infosys Springboard internship.

My role is Frontend Development and API Integration. I build the pages that Quality Engineers and Factory Supervisors will actually use — uploading images, viewing results, checking quality trends — and connect these pages to the backend once it's ready.

2. Tech Stack & Environment Setup

I built this using VS Code, along with:

- React (with Vite) – for building the pages
- Tailwind CSS – for styling
- React Router – for switching between pages
- Backend Integration – Rest API'S
- React Context API – for remembering who's logged in
- JWT Authentication – to connect securely to the backend's login
- Backend API Server – Currently integrated with the local backend running on <http://localhost:5000>.
- Git & GitHub – for saving my work on my own branch (HimabindhuRavuri)

3. Mile Stone 2 Work

For Milestone 2, my main focus was integrating the frontend with the backend and preparing the application for AI model integration.

The main tasks I worked on were:

- Connecting the login page with the backend login API
- Implementing JWT-based authentication
- Adding role-based access to the application
- Connecting the History page with the backend API
- Connecting the Analytics page with the backend APIs
- Preparing the Inspection page for image upload API integration
- Creating the Reports page
- Updating the navigation based on user roles
- Adding logout functionality
- Testing frontend and backend communication

4. Authentication and Role-Based Access

I connected the frontend login page with the backend login API:

POST /api/auth/login

After successful login, the backend sends an access token and user information. The frontend stores the user information and uses the access token when calling protected APIs.

I also implemented role-based access in the frontend. The application currently has three user roles, and each role has access to different features.

Faculty Inspector

- Dashboard
- Inspection
- History

Quality Engineer

- Dashboard
- Inspection
- History
- Analytics

Admin

- Dashboard
- Inspection
- History
- Analytics
- Admin

This allows each user to access only the pages and features required for their role.

5. Inspection Module

I worked on the Inspection page where users can upload product images for inspection.

The frontend is connected to the inspection API:

POST /api/inspection/image

The image is sent to the backend using multipart/form-data.

The backend is expected to process the image and communicate with the AI model. The response contains details such as:

- Inspection status
- AI score
- Inspection ID
- File name
- User ID

The frontend side of the image upload and API integration has been prepared. The final step is to connect and test the complete workflow with the AI model and display the actual prediction results.

6. Inspection History

I worked on the History page to display the previous inspection records of the logged-in user.

The frontend is connected to:

GET /api/history?limit=50&offset=0

The History page is designed to show inspection records received from the backend.

I also worked on search and filtering options so that users can easily find specific inspection results.

The page is currently being tested and finalized with real backend data.

7. Analytics

I worked on the Analytics page to display inspection-related information and quality trends.

The frontend is connected to the following APIs:

GET /api/analytics

GET /api/analytics/by-status

These APIs are used to get analytics information and inspection results based on their status.

The Analytics page is available only to roles that have permission to view analytics, such as Quality Engineers and Production Managers.

8. Reports

I created a Reports page as part of the frontend application.

Currently, there is no separate Reports API available in the backend. So, the Reports page is planned to use the available analytics data.

The Reports page can be improved later by adding features such as downloading reports or generating PDF reports once the required backend functionality is available.

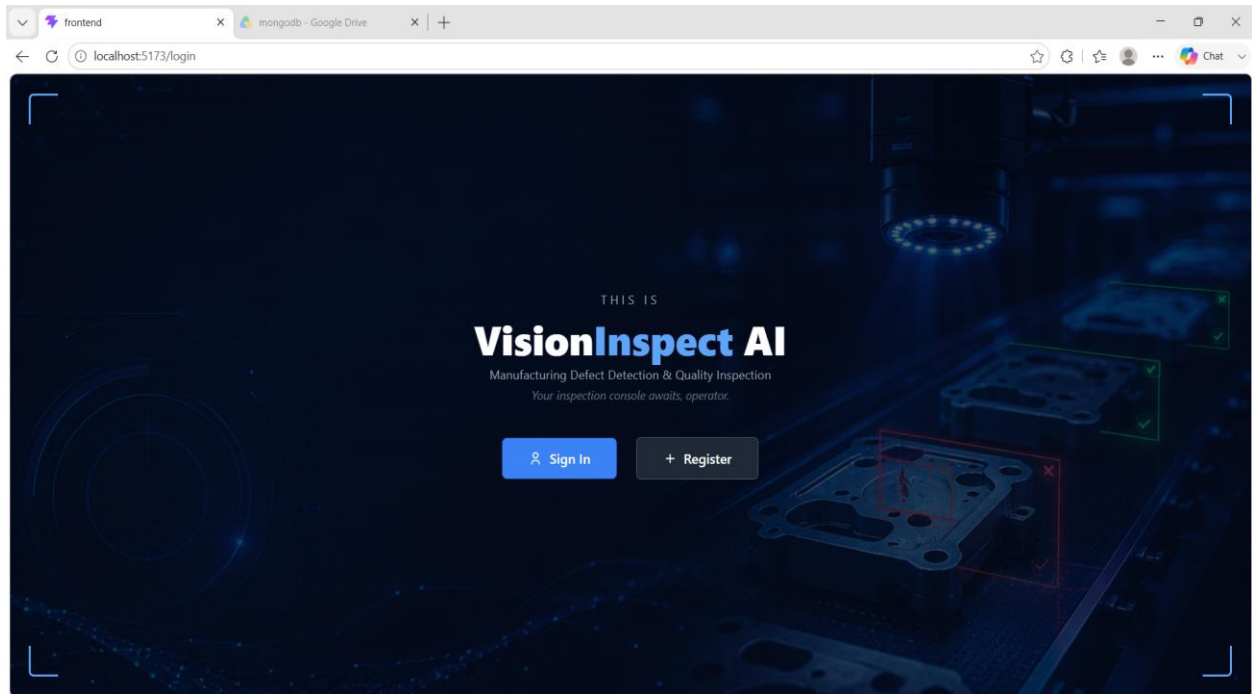
9. API Integration

The main APIs I worked with during Milestone 2 are:

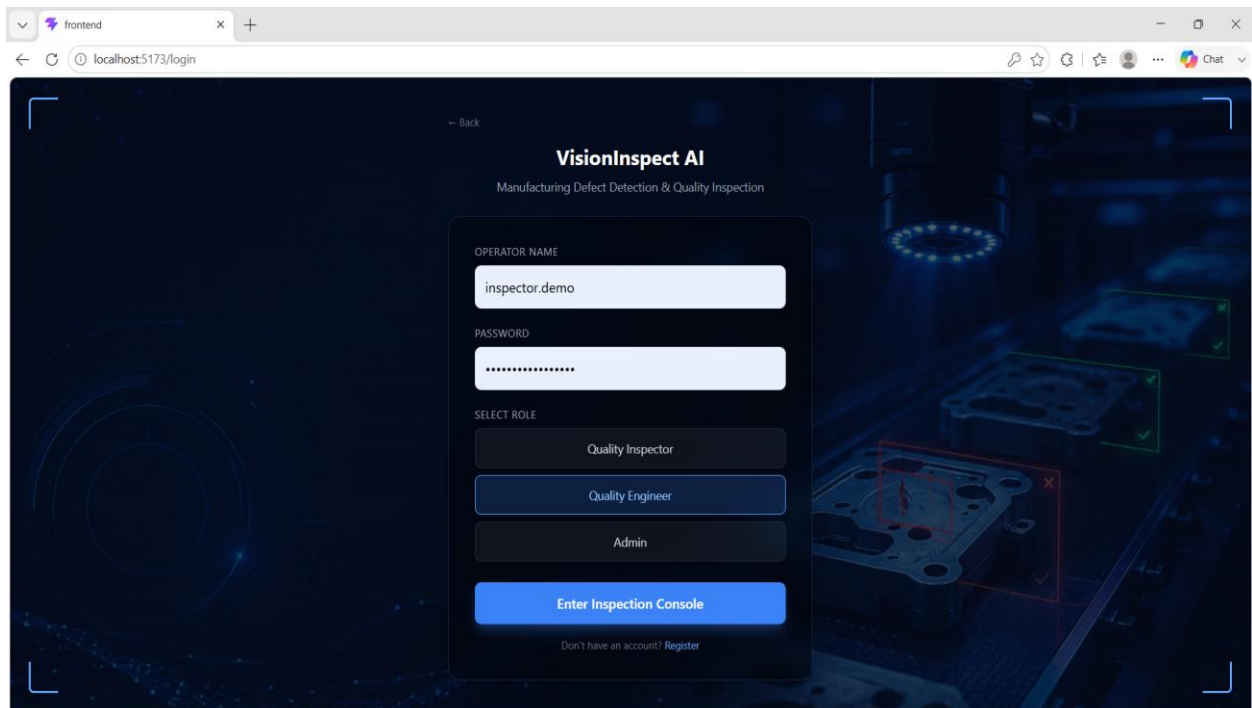
API METHOD	ENDPOINT	PURPOSE
POST	/api/auth/login	User login and authentication
GET	/api/history	Get inspection history
GET	/api/analytics	Get inspection analytics
GET	/api/analytics/by-status	Get analytics based on inspection status
POST	/api/inspection/image	Upload image for inspection

10. Screenshots

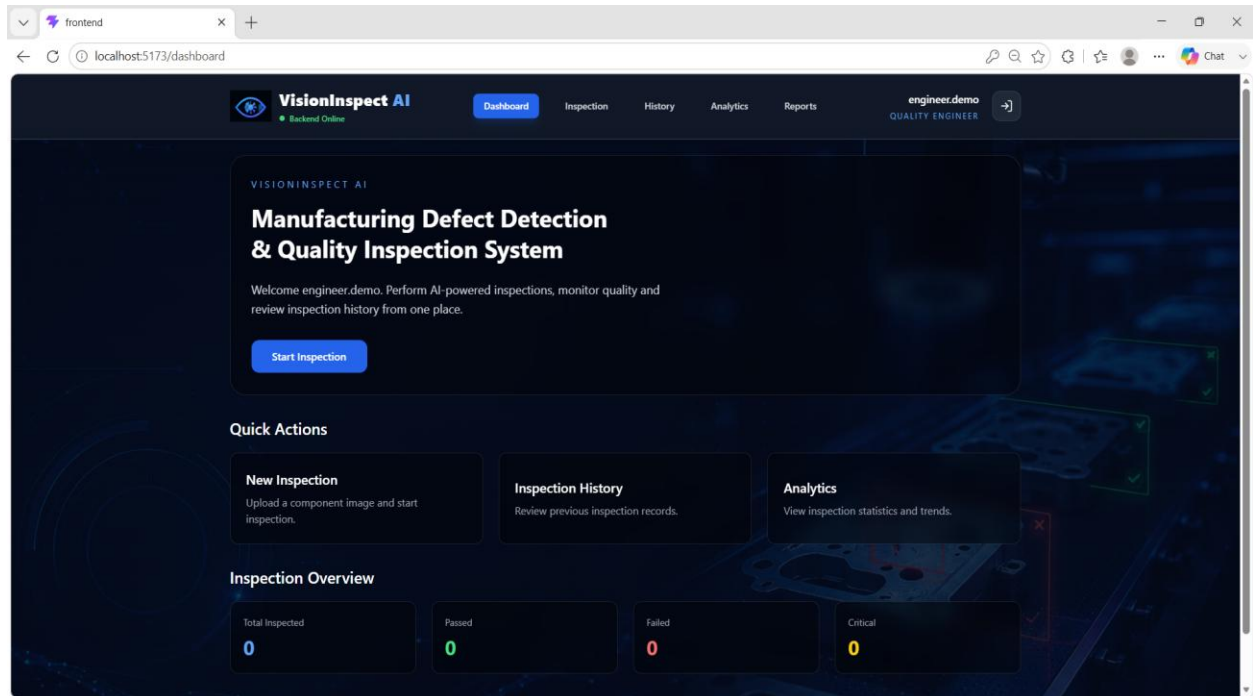
Login Page



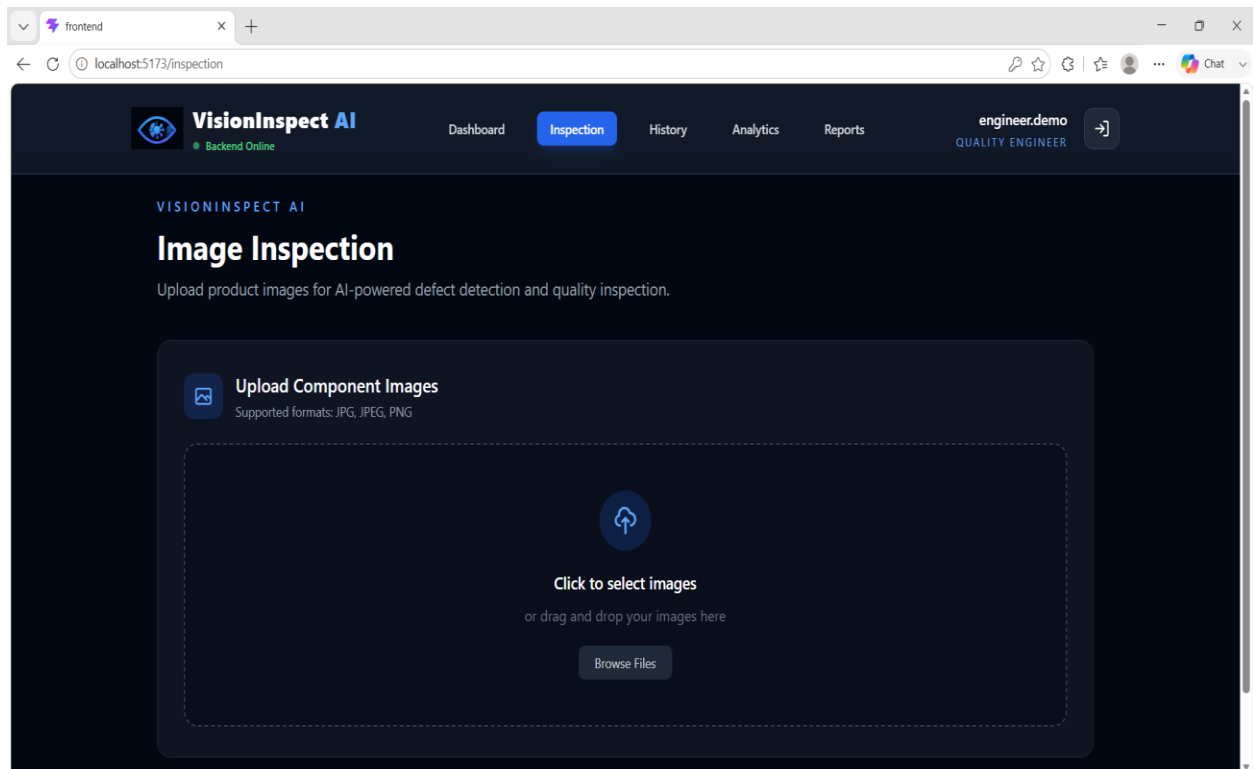
Role Based Access



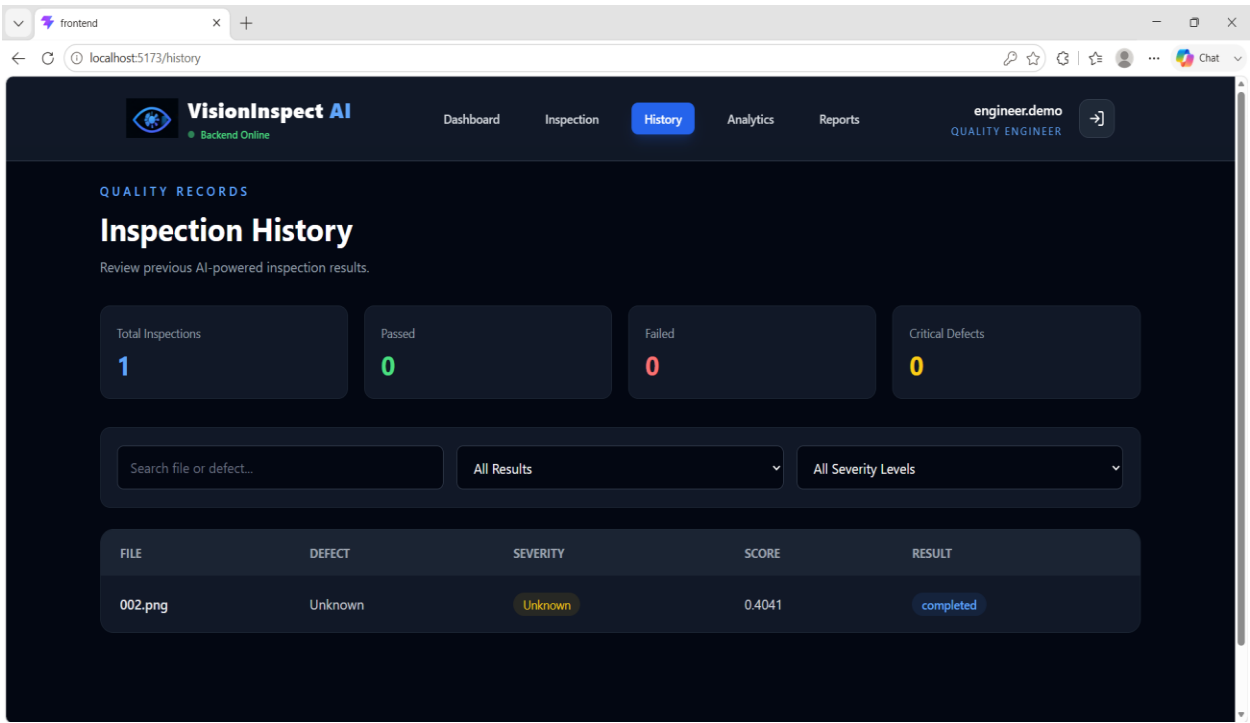
Dashboard



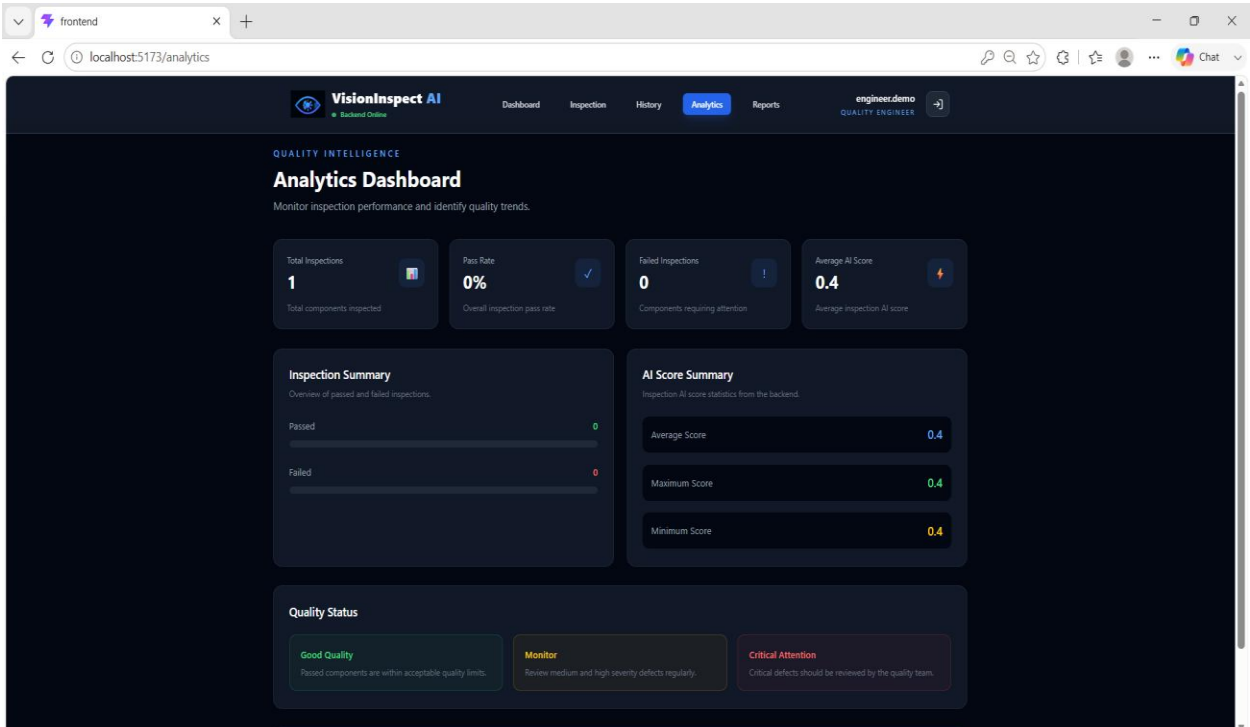
Inspection Page



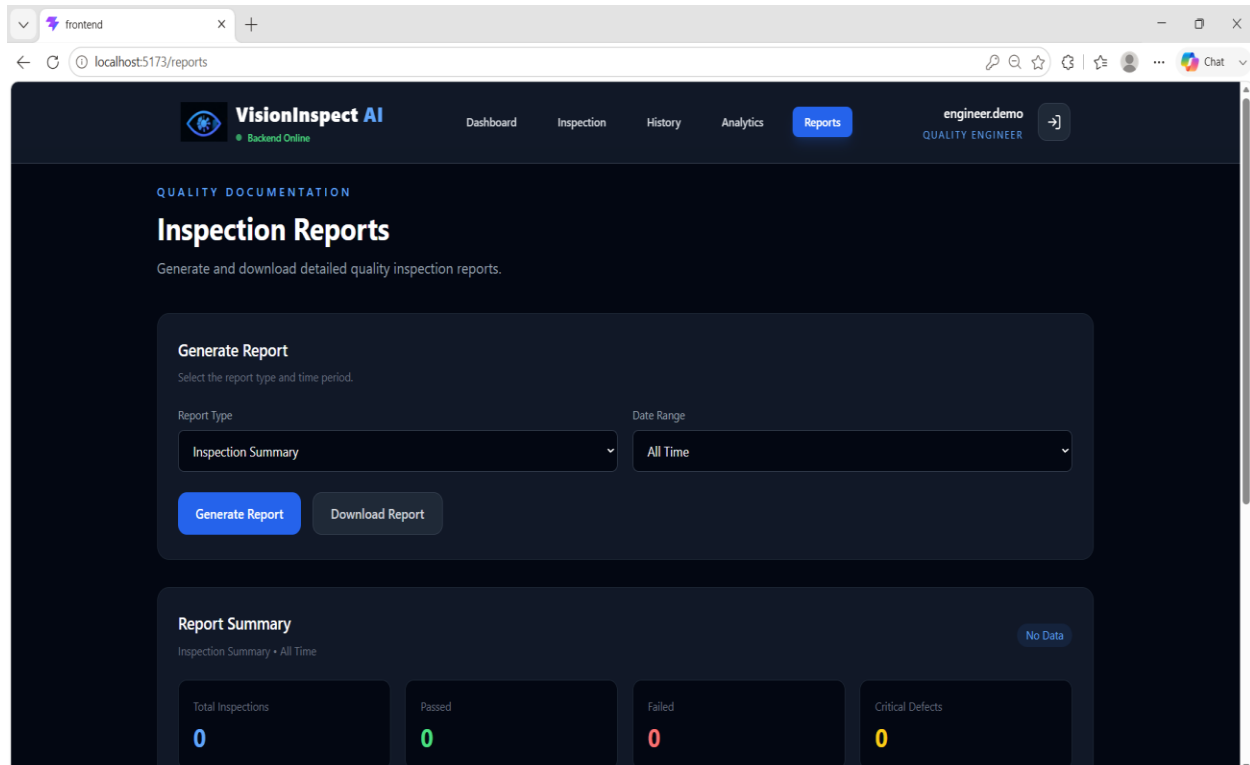
History Page



Analytics Page



Reports Page



11. Current Progress

Till now, I have completed the following work:

- Frontend setup using React, Vite, and Tailwind CSS
- Login and registration pages
- JWT-based login integration
- Role-based access control
- Role-based navigation
- Dashboard
- Inspection page, Inspection image API integration preparation
- History page, History API integration
- Analytics page, Analytics API integration
- Reports page
- Logout functionality
- Frontend and backend integration

12. Remaining Work

The remaining work includes:

- Complete AI model integration with the backend
- Test the complete image inspection workflow
- Display real AI prediction results in the frontend
- Complete testing of the History page with live data
- Replace any remaining temporary data with real API data
- Test all user roles and their permissions
- Test error handling and invalid inputs
- Perform complete frontend, backend, and AI integration testing

13. Conclusion

During Milestone 2, I mainly focused on connecting the frontend application with the backend APIs and making the application work with real backend data.

I implemented authentication, role-based access, inspection API integration, inspection history, analytics, and reports. I also prepared the Inspection page for integration with the AI model.

The main remaining task is to complete the connection between the backend and AI model and test the complete inspection process from uploading an image to displaying the final AI result.